



AUDIT REPORT

February 2026

For

fluxion

Table of Content

Executive Summary	03
Number of Security Issues per Severity	06
Summary of Issues	07
Checked Vulnerabilities	09
Techniques and Methods	10
Types of Severity	12
Types of Issues	13
Severity Matrix	14
 Informational Issues	15
1. Magic Numbers Used for Fee Tiers and Tick Spacing	15
Functional Tests	16
Automated Tests	19
Threat Model	20
Closing Summary & Disclaimer	45



Executive Summary

Project Name Fluxion-Exchange

Protocol Type AMM

Project URL <https://www.fluxion.exchange/>

Overview Fluxion-Exchange V3 is a decentralized exchange (DEX) protocol that closely follows the architecture and design philosophy of Uniswap V3, with both its core and periphery components modeled after the Uniswap V3 implementation. The core contracts define the fundamental AMM logic, including liquidity pools, token pair management, swap execution, fee accounting, and tick-based mechanisms that enable concentrated liquidity. This design allows liquidity providers to supply capital within custom price ranges, improving capital efficiency compared to traditional constant-product AMMs. The periphery layer consists of auxiliary and convenience contracts such as routers, position managers, and multicall utilities that simplify user interactions with the core system. These contracts abstract away low-level pool logic and provide higher-level functions for swapping tokens, minting or adjusting liquidity positions, and batching multiple actions into a single transaction. By separating immutable protocol logic (core) from user-facing helpers (periphery), Fluxion-Exchange V3 maintains a clean, modular structure that mirrors Uniswap V3's proven model. Overall, the project can be viewed as a Uniswap V3-style fork or derivative, reusing its core and periphery patterns to deliver a concentrated-liquidity AMM with familiar behavior and tooling.

Audit Scope The scope of this Audit was to analyze the Fluxion-Exchange Smart Contracts for quality, security, and correctness.

Source Code link <https://github.com/Fluxion-Exchange/v3-core>

Contracts in Scope

- BlockTimestamp.sol
- ERC721Permit.sol
- LiquidityManagement.sol
- Multicall.sol
- PeripheryImmutableState.sol
- PeripheryPayments.sol



PeripheryPaymentsWithFee.sol
PeripheryValidation.sol
PoolInitializer.sol
SelfPermit.sol
BytesLib.sol
CallbackValidation.sol
ChainId.sol
HexStrings.sol
LiquidityAmounts.sol
NFTDescriptor.sol
NFTSVG.sol
OracleLibrary.sol
Path.sol
PoolAddress.sol
PoolTicksCounter.sol
PositionKey.sol
PositionValue.sol
SqrtPriceMathPartial.sol
TokenRatioSortOrder.sol
TransferHelper.sol
NonfungiblePositionManager.sol
SwapRouter.sol
NoDelegateCall.sol
UniswapV3Factory.sol
UniswapV3Pool.sol
UniswapV3PoolDeployer.sol
BitMath.sol
FixedPoint128.sol
FixedPoint96.sol
FullMath.sol
LiquidityMath.sol
LowGasSafeMath.sol
Oracle.sol
Position.sol
SafeCast.sol
SqrtPriceMath.sol
SwapMath.sol
Tick.sol
TickBitmap.sol
TickMath.sol
UnsafeMath.sol

Branch

main

Commit Hash

2e7e79c0cb333764092218c03089ab4094656bae

Language

Solidity



Blockchain	Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	9th February to 12th February

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0(0.0%)
High	0(0.0%)
Medium	0(0.0%)
Low	0(0.0%)
Informational	1 (100.0%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	1
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	0	0	0



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Magic Numbers Used for Fee Tiers and Tick Spacing	Informational	Acknowledged



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Informational Issues

Magic Numbers Used for Fee Tiers and Tick Spacing

Acknowledged

Description

The contract uses hardcoded numeric values (e.g., 500, 3000, 10000, 10, 60, 200) when initializing fee tiers and corresponding tick spacing. These values are directly embedded in the logic without being defined as named constants. While functionally correct, the use of magic numbers reduces code readability, maintainability, and clarity. Future modifications may introduce inconsistencies if these values are reused or updated without a centralized definition.

Impact

Low

Likelihood

Low

Recommendation

Define the fee tiers and tick spacing values as clearly named constant variables to improve readability, maintainability, and reduce the risk of misconfiguration during future updates.

Functional Tests

Some of the tests performed are mentioned below:

UniswapV3Factory

- ✓ Factory deploys successfully with deployer assigned as initial contract owner address.
- ✓ Default fee tiers are initialized with their corresponding tick spacing values correctly.
- ✓ Pool creation succeeds when valid token pair and enabled fee tier are provided.
- ✓ Pool creation reverts when both provided token addresses are identical.
- ✓ Pool creation reverts when pool already exists for token pair and fee.
- ✓ Token addresses are automatically sorted to maintain canonical token0 ordering.
- ✓ Multiple pools can be created for same pair using different fee tiers.
- ✓ Contract owner can enable new fee tiers with valid parameters.
- ✓ Enabling a fee tier greater than or equal to maximum value reverts.
- ✓ Ownership transfer attempts by non owner correctly revert.
- ✓ PoolCreated event emits correct token addresses fee and pool address.
- ✓ FeeAmountEnabled event emits correct fee amount and tick spacing.

UniswapV3Factory

Initialization

- ✓ Pool initializes successfully with valid sqrtPriceX96 and starting tick value.
- ✓ Attempting to initialize an already initialized pool correctly reverts.
- ✓ Initialization with sqrtPriceX96 below minimum allowed value reverts.
- ✓ Initialization with sqrtPriceX96 at or above maximum allowed value reverts.



Minting Liquidity

- ✓ Mint creates a new liquidity position within the specified tick range.
- ✓ Both tokens are deposited when minting inside current price range.
- ✓ Only token0 is deposited when minting fully below price range.
- ✓ Only token1 is deposited when minting fully above price range.
- ✓ Position state is updated correctly after successful mint operation.
- ✓ Mint reverts when tickLower is greater than or equal to tickUpper.
- ✓ Mint reverts when ticks are not aligned to tick spacing.
- ✓ Mint reverts when zero liquidity amount is specified.
- ✓ Tick is initialized when liquidity is added for first time.
- ✓ Global pool liquidity increases when minting inside price range.

Burning Liquidity

- ✓ Burn decreases position liquidity and increases owed token balances correctly.
- ✓ Burning more liquidity than available position balance correctly reverts.
- ✓ Tick is cleared when last liquidity is removed from it.
- ✓ Global pool liquidity decreases when burning inside price range.
- ✓ Accrued fees are calculated and added to tokens owed.

Collecting

- ✓ Collect transfers correct token0 and token1 amounts to recipient.
- ✓ Collect reduces tokens owed balances by collected amounts.
- ✓ Maximum collect parameters retrieve all available owed tokens,

Swapping

- ✓ Exact input swap exchanges specified input amount for output tokens.
- ✓ Exact output swap exchanges for exact desired output amount.
- ✓ Swap correctly handles token0 to token1 direction.
- ✓ Swap correctly handles token1 to token0 direction.
- ✓ Swap callback is invoked with correct token delta amounts.
- ✓ Slot0 values updated with new price and tick after swap.



- ✓ Swap respects provided `sqrtPriceLimitX96` parameter.
- ✓ Large swaps correctly cross multiple initialized tick boundaries.
- ✓ Liquidity is updated when crossing initialized ticks.
- ✓ Fees are calculated and accumulated in global fee variables.
- ✓ Swap with insufficient liquidity correctly reverts.
- ✓ Protocol fees are accumulated when protocol fee is enabled.

Flash

- ✓ Flash loan transfers requested token amounts to recipient correctly.
- ✓ Flash callback invoked with correct amounts and calculated fees.
- ✓ Flash reverts when borrowed tokens and fees are not repaid.

Fee Management

- ✓ Swap fees distributed to liquidity providers proportionally.
- ✓ Global fee growth variables increase after each swap.

Tick Management

- ✓ Ticks initialize with correct liquidity and fee growth values.
- ✓ Crossing a tick updates pool liquidity correctly.
- ✓ `nextInitializedTickWithinOneWord` finds correct next initialized tick.

UniswapV3PoolDeployer

- ✓ Pool deployed at deterministic CREATE2 address.
- ✓ Deployment parameters populated before pool creation.
- ✓ Deployment parameters cleared after pool creation.

SwapRouter

- ✓ `exactInputSingle` swaps exact input amount correctly.
- ✓ `exactOutputSingle` swaps for exact output amount correctly.
- ✓ Single swap transfers input token from caller.



- ✓ Single swap transfers output token to recipient.
- ✓ Swap enforces amountOutMinimum slippage protection.
- ✓ Swap with expired deadline correctly reverts.
- ✓ Swap respects provided sqrtPriceLimitX96 parameter.
- ✓ exactInput performs multi hop swap correctly.
- ✓ exactOutput performs multi hop swap correctly.
- ✓ Multi hop uses correct fee tiers from path.
- ✓ Multi hop reverts if pool does not exist.
- ✓ Callback validates caller is legitimate pool.
- ✓ Callback reverts when caller is not pool.
- ✓ ETH input swaps are wrapped to WETH automatically.
- ✓ ETH output swaps are unwrapped from WETH automatically.
- ✓ refundETH returns excess ETH to caller.
- ✓ unwrapWETH9 sends ETH to recipient.
- ✓ multicall executes multiple operations in single transaction.
- ✓ multicall reverts entire batch if any call fails.
- ✓ All swap functions validate provided deadline.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC20.sol	Library / Base Contract	Core token logic balances, allowances, transfer mechanics inherited by any ERC-20 paired in a pool	Correct implementation of ERC-20 invariants; no side-effects on transfer	Fee-on-transfer and rebasing tokens silently break pool accounting; malicious transfer hooks enable reentrancy
ERC20Detailed.sol	Library / Metadata	Token name, symbol, and decimals exposed for UI display and off-chain tooling	Metadata is static, honest, and returns expected types	N/A
IERC20Minimal	Interface	UniswapV3Pool calls balanceOf() via staticcall to verify payment after callbacks; periphery uses transfer / transferFrom	balanceOf faithfully reflects on-chain balance; transfer return value is reliable	N/A



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
WETH9 (Wrapped ETH)	External Contract	PeripheryPayment.s.sol wraps ETH→WETH before swaps & unwraps WETH→ETH for recipients using an immutable address set at deploy time	Canonical, immutable WETH9 correctly implements ERC-20 + deposit() / withdraw()	Incorrectly deployed WETH9 drains all ETH-paired swaps; no upgrade path if WETH9 is exploited
Solidity Compiler (≤0.7.x)	Toolchain	All contracts compiled with pragmas in >=0.5.0 <0.8.0; optimizer enabled	Compiler version has no critical code-generation bugs; ABI encoding is deterministic	Pre-0.8 has no built-in checked arithmetic – overflow protection must be manual; optimizer bugs in old versions can silently miscompile math-heavy code
ERC-721 (OpenZeppelin)	Library	NonfungiblePositionManager inherits OZ ERC-721 for minting, transferring, and burning LP-position tokens	OZ ERC-721 is correct, unmodified, and free of known vulnerabilities	Any OZ ERC-721 vulnerability compromises all LP positions; transferring the NFT transfers full liquidity control
SafeERC20 (OpenZeppelin)	Library	Periphery wraps all token operations to handle tokens that return false instead of reverting	OZ SafeERC20 correctly handles non-standard tokens and checks return data	Tokens without ERC-20 return values (e.g. USDT on mainnet) would fail without this wrapper; incorrect wrapping could mask failed transfers



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
SelfPermit (Periphery Base)	Base Contract	Enables gasless ERC-20 approvals via EIP-2612 permit() before a swap or liquidity op in one transaction	Token supports EIP-2612; permit signature is fresh and single-use	Signature replay if token doesn't enforce nonces; front-running the permit call to drain approval
block.timestamp	EVM Primitive	Oracle observations timestamped as uint32(block.timestamp); periphery deadline checks compare against caller-supplied deadline	Validators cannot skew timestamp beyond ~15 seconds; uint32 wrap known and accepted (year ~2106)	Miner timestamp manipulation biases TWAP; uint32 overflow breaks oracle in 2106; generous deadlines expose users to MEV sandwich attacks
Ethereum L2 / Fluxion Network	Infrastructure	All contracts deployed on the Fluxion Network chain; full EVM compatibility assumed	Host chain provides correct EVM semantics, block finality, and event logs	Non-standard EVM behaviour (different BLOCKHASH, non-sequential timestamps) can break oracle and deadline logic; finality delays expose users to reorg risk



2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
UniswapV3Factory .sol	createPool(address tokenA, address tokenB, uint24 fee)	What this function can do Deploy a new UniswapV3Pool contract for the token pair and fee. Automatically order tokens (token0 < token1). Store pool address in bidirectional mapping. Use CREATE2 for deterministic pool addresses. Emit PoolCreated event. What this function cannot/should not do Cannot create a pool if one already exists for the token pair and fee. Cannot create a pool with identical tokens. Cannot create a pool with invalid/zero addresses. Cannot create a pool with unsupported fee tier. Cannot initialize the pool's price. Main invariant(s) Each (token0, token1, fee) combination has exactly one pool. Pool address is deterministic based on CREATE2. token0 < token1 always. Bidirectional mapping consistency maintained. Access level Public/External (permissionless)
UniswapV3Factory .sol	setOwner(address _owner)	What this function can do Transfer ownership of the factory to a new address. Emit OwnerChanged event with old and new owner. Allow new owner to control protocol fees and fee tiers.

Contract Name	Function	Category
UniswapV3Factory.sol	initialize(uint160 sqrtPriceX96)	What this function cannot/should not do Cannot be called by non-owner addresses. Should not set owner to address(0). Cannot transfer ownership to existing owner. Cannot retroactively affect already deployed pools. Main invariant(s) Only current owner can transfer ownership. Owner change is immediate and atomic. Exactly one owner exists at any time. Previous owner loses all privileges immediately. Access level Restricted (onlyOwner modifier) What this function can do Set the initial price of the pool. Calculate and set the initial tick. Enable pool for trading and liquidity provision. Emit Initialize event. What this function cannot/should not do Cannot be called if pool is already initialized. Cannot be called multiple times. Cannot initialize with invalid price ($\text{sqrtPriceX96} = 0$). Cannot initialize with price outside valid range. Cannot add liquidity during initialization. Main invariant(s) Pool can be initialized exactly once. sqrtPriceX96 must be within valid tick range. After initialization, $\text{sqrtPriceX96} \neq 0$. Initial tick matches the sqrtPrice. Initialization is permanent and irreversible. Access level Public/External (permissionless, but only effective once)

Contract Name	Function	Category
UniswapV3Pool.sol	mint(address recipient, int24 tickLower, int24 tickUpper, uint128 amount, bytes calldata data)	What this function can do Add liquidity to the pool within specified price range. Update position state for recipient. Update tick liquidity gross and net values. Transfer tokens from sender via callback. Collect fees earned in the range. Emit Mint event. What this function cannot/should not do Cannot mint if pool is not initialized. Cannot mint with tickLower $\geq$ tickUpper. Cannot mint with ticks not aligned to tickSpacing. Cannot mint with amount = 0. Cannot mint outside valid tick range. Cannot bypass token transfer callback verification. Main invariant(s) Pool must be initialized before minting. tickLower < tickUpper. Ticks must be multiples of tickSpacing. Position liquidity increases by specified amount. Token balances increase by exact amounts returned. Protected by reentrancy lock. Access level Public/External (requires valid callback implementation)
UniswapV3Pool.sol	burn(int24 tickLower, int24 tickUpper, uint128 amount)	What this function can do Remove liquidity from a position. Update position and tick state. Calculate tokens owed to position owner. Emit Burn event. Return amounts of token0 and token1 to collect. What this function cannot/should not do Cannot burn more liquidity than position has. Cannot burn if ticks are invalid or misaligned. Cannot burn with amount = 0. Does not transfer tokens (must call collect separately). Cannot burn other users' positions.

Contract Name	Function	Category
UniswapV3Pool.sol	collect(address recipient, int24 tickLower, int24 tickUpper, uint128 amount0Requested, uint128 amount1Requested)	Main invariant(s) Can only burn liquidity owned by msg.sender. Burned amount $\leq$ position liquidity. Position liquidity decreases by exact amount burned. Tick liquidity updated correctly. Tokens remain in pool until collect() is called. Protected by reentrancy lock. Access level Public/External (can only burn own positions) What this function can do Collect tokens owed from burned liquidity and fees. Transfer token0 and token1 to recipient. Update position's tokensOwed values. Emit Collect event. Return actual amounts collected. What this function cannot/should not do Cannot collect more than tokensOwed for the position. Cannot collect from other users' positions. Cannot collect if ticks are invalid. Should not collect to address(0). Cannot bypass balance checks. Main invariant(s) Can only collect from own position. Collected amount $\leq$ min(requested, tokensOwed). tokensOwed decreases by collected amount. Pool balances decrease by collected amounts. Protected by reentrancy lock. Access level Public/External (can only collect own position tokens)

Contract Name	Function	Category
UniswapV3Pool.sol	swap(address recipient, bool zeroForOne, int256 amountSpecified, uint160 sqrtPriceLimitX96, bytes calldata data)	What this function can do Execute token swap within the pool. Move price and current tick. Cross ticks and update liquidity. Collect swap fees. Transfer tokens via callback. Emit Swap event. Update oracle observations. What this function cannot/should not do Cannot swap if pool not initialized. Cannot swap with invalid price limit. Cannot swap with amountSpecified = 0. Cannot swap beyond price limit. Cannot bypass callback verification. Should not allow re-entrancy. Cannot swap if insufficient liquidity. Main invariant(s) Pool must be initialized. Price moves in expected direction based on zeroForOne. Cannot exceed sqrtPriceLimitX96. Swap fee is properly collected. Token balance changes match swap amounts. Virtual liquidity is consistent after swap. Oracle is updated. Protected by reentrancy lock. Access level Public/External (requires valid callback implementation)
UniswapV3Pool.sol	flash(address recipient, uint256 amount0, uint256 amount1, bytes calldata data)	What this function can do Provide flash loan of token0 and/or token1. Transfer tokens to recipient. Execute callback to recipient. Collect flash fee. Verify tokens returned plus fee. Emit Flash event. What this function cannot/should not do Cannot flash more than pool's token balance. Cannot skip fee collection. Cannot allow re-entrancy. Should not flash to address(0). Cannot bypass balance verification after callback.

Contract Name	Function	Category
UniswapV3Pool.sol	setFeeProtocol(uint8 feeProtocol0, uint8 feeProtocol1)	Main invariant(s) Flash amounts $\leq$ pool balances. Tokens + fees must be returned in same transaction. Pool balances $\geq$ original balance after flash. Flash fee properly collected and distributed. Protected by reentrancy lock. No net loss of tokens from pool. Access level Public/External (requires valid callback that returns tokens + fee) What this function can do Set protocol fee share for token0 and token1. Update slot0 with new fee protocol values. Emit SetFeeProtocol event. What this function cannot/should not do Cannot be called by anyone except factory owner. Cannot set fee protocol values > 10. Cannot set asymmetric fees that harm LPs. Should not be called frequently (gas consideration). Main invariant(s) Only factory owner can set fee protocol. Fee protocol values $\in [0, 10]$ representing 0-100% in increments of 10%. Fee split: LP gets (denominator - feeProtocol)/denominator. Both fees can be set independently. Access level Restricted (only factory.owner())
UniswapV3Pool.sol	collectProtocol(address recipient, uint128 amount0Requested, uint128 amount1Requested)	What this function can do Collect accumulated protocol fees. Transfer fees to recipient address. Update protocol fee accumulator. Emit CollectProtocol event. Return actual amounts collected.

Contract Name	Function	Category
UniswapV3Pool.sol	increaseObservationCardinalityNext(uint16 observationCardinalityNext)	What this function cannot/should not do Cannot be called by anyone except factory owner. Cannot collect more than accumulated fees. Should not collect to address(0). Cannot bypass balance checks. Main invariant(s) Only factory owner can collect protocol fees. Collected amount $\leq$ min(requested, accumulated). Protocol fee accumulator decreases by collected amount. Pool balances decrease appropriately. Access level Restricted (only factory.owner()) What this function can do Increase the maximum number of price observations stored. Allow for longer-term TWAP calculations. Emit IncreaseObservationCardinalityNext event. Pay for storage expansion. What this function cannot/should not do Cannot decrease observation cardinality. Cannot set cardinality lower than current. Cannot exceed uint16 maximum. Expensive operation (pays for SSTORE). Main invariant(s) Cardinality can only increase. New cardinality $\geq$ current cardinality. Observation array expands to accommodate. Anyone can call and pay for expansion. Access level Public/External (permissionless, caller pays gas)

Contract Name	Function	Category
NonfungiblePositionManager.sol	mint(MintParams calldata params)	What this function can do Create new liquidity position as ERC721 NFT. Add liquidity to pool within specified range. Transfer tokens from user via permit or transferFrom. Mint NFT representing position to recipient. Store position data (liquidity, tick range, tokens, fee). Emit IncreaseLiquidity event. What this function cannot/should not do Cannot mint with deadline expired. Cannot mint with slippage exceeded (amount0Min/amount1Min). Cannot mint to address(0). Cannot mint with invalid tick range. Should not allow re-entrancy. Cannot bypass token approvals. Main invariant(s) Each tokenId represents unique position. NFT owner controls the liquidity position. Position data immutably stored (pool, tickLower, tickUpper, fee). Liquidity matches pool state. Token amounts meet minimum requirements. Protected by reentrancy lock. Access level Public/External (requires token approvals)
NonfungiblePositionManager.sol	increaseLiquidity(IncreaseLiquidityParams calldata params)	What this function can do Add more liquidity to existing position. Transfer additional tokens from user. Update position liquidity amount. Emit IncreaseLiquidity event. Return updated liquidity and token amounts. What this function cannot/should not do Cannot increase liquidity for non-existent tokenId. Cannot be called by non-approved operator. Cannot increase with deadline expired. Cannot increase with slippage exceeded. Cannot increase liquidity by 0.



Contract Name	Function	Category
NonfungiblePositionManager.sol	decreaseLiquidity(DecreaseLiquidityParams calldata params)	Main invariant(s) Only NFT owner or approved operator can increase. Position's tick range remains unchanged. Liquidity increases monotonically. Token amounts meet minimums. Protected by authorization checks. Access level Restricted (requires NFT ownership or approval) What this function can do Remove liquidity from existing position. Burn liquidity in pool. Update position state. Track tokens owed to position. Emit DecreaseLiquidity event. Return amounts of tokens decremented. What this function cannot/should not do Cannot decrease more liquidity than position has. Cannot be called by unauthorized user. Cannot decrease with deadline expired. Does not transfer tokens (must call collect). Cannot decrease non-existent position. Main invariant(s) Only NFT owner or approved operator can decrease. Decreased amount $\leq$ position liquidity. Liquidity decreases correctly. Tokens owed increases by amounts from burn. Position remains valid even with 0 liquidity. Protected by authorization checks. Access level Restricted (requires NFT ownership or approval)
NonfungiblePositionManager.sol	collect(CollectParams calldata params)	What this function can do Collect tokens owed from decreased liquidity and fees. Transfer tokens to recipient. Update position's tokensOwed tracking. Emit Collect event. Return amounts collected.



Contract Name	Function	Category
NonfungiblePositionManager.sol	burn(uint256 tokenId)	What this function cannot/should not do Cannot collect more than owed. Cannot be called by unauthorized user. Cannot collect from non-existent position. Should not collect to address(0). Cannot bypass max amount limits. Main invariant(s) Only NFT owner or approved operator can collect. Collected amount $\leq$ min(requested, owed). Tokens owed decreases by collected amount. Tokens transferred to specified recipient. Protected by authorization checks. Access level Restricted (requires NFT ownership or approval) What this function can do Burn/destroy NFT representing position. Remove position from storage. Emit Transfer event to address(0). Free up storage (gas refund). What this function cannot/should not do Cannot burn position with remaining liquidity. Cannot burn position with uncollected tokens. Cannot burn non-existent tokenId. Cannot be called by non-approved operator. Cannot burn position owned by others. Main invariant(s) Can only burn if liquidity = 0. Can only burn if tokensOwed0 = 0 and tokensOwed1 = 0. Only NFT owner or approved operator can burn. NFT is permanently destroyed. TokenId cannot be reused. Access level Restricted (requires NFT ownership or approval)

Contract Name	Function	Category
NonfungiblePositionManager.sol	createAndInitializePoolIfNecessary(address token0, address token1, uint24 fee, uint160 sqrtPriceX96)	What this function can do Check if pool exists for token pair and fee. Create pool via factory if it doesn't exist. Initialize pool with starting price. Return pool address. What this function cannot/should not do Cannot create pool with tokens in wrong order (must be sorted). Cannot initialize already initialized pool. Cannot create pool with invalid fee tier. Cannot initialize with invalid sqrtPrice. Main invariant(s) If pool exists, returns existing address without changes. If pool doesn't exist, creates and initializes it. Pool is always initialized after function completes. Idempotent operation (safe to call multiple times). Access level Public/External (permissionless helper function)
SwapRouter.sol	exactInputSingle (ExactInputSingleParams calldata params)	What this function can do Execute single-pool swap with exact input amount. Transfer input tokens from user. Perform swap on specified pool. Transfer output tokens to recipient. Enforce minimum output amount (slippage protection). Refund unused input tokens if any. What this function cannot/should not do Cannot execute if deadline expired. Cannot execute if output < amountOutMinimum. Cannot swap with sqrtPriceLimitX96 in wrong direction. Cannot bypass token approvals. Should not allow re-entrancy.

Contract Name	Function	Category
SwapRouter.sol	exactInput(Exact InputParams calldata params)	Main invariant(s) Must complete before deadline. Actual output $\geq$ amountOutMinimum. Input amount is exact (as specified). Protected by deadline check. Protected by slippage check. Protected by reentrancy lock. Access level Public/External (requires token approvals) What this function can do Execute multi-hop swap with exact input amount. Route through multiple pools sequentially. Transfer input tokens from user. Perform sequential swaps through path. Transfer final output tokens to recipient. Enforce minimum output amount. What this function cannot/should not do Cannot execute if deadline expired. Cannot execute if final output $<$ amountOutMinimum. Cannot use invalid path encoding. Cannot skip intermediate pools. Cannot bypass token approvals for first hop. Main invariant(s) Path must have valid encoding (token-fee-token-fee-...). Final output $\geq$ amountOutMinimum. Input amount is exact (as specified). Each hop executes correctly in sequence. Protected by deadline and slippage checks. Protected by reentrancy lock. Access level Public/External (requires token approvals)

Contract Name	Function	Category
SwapRouter.sol	exactOutputSingle(ExactOutputSingleParams calldata params)	What this function can do Execute single-pool swap with exact output amount. Calculate and transfer required input tokens. Perform swap on specified pool. Transfer exact output tokens to recipient. Enforce maximum input amount (slippage protection). Refund excess input tokens to sender. What this function cannot/should not do Cannot execute if deadline expired. Cannot execute if required input > amountInMaximum. Cannot swap with invalid price limit. Cannot bypass token approvals. Should not allow re-entrancy. Main invariant(s) Must complete before deadline. Actual input $\leq$ amountInMaximum. Output amount is exact (as specified). Excess input tokens refunded. Protected by deadline check. Protected by slippage check. Protected by reentrancy lock. Access level Public/External (requires token approvals)
SwapRouter.sol	exactOutput(ExactOutputParams calldata params)	What this function can do Execute multi-hop swap with exact output amount. Route through multiple pools in reverse order. Calculate and transfer required input tokens. Perform sequential swaps through reverse path. Transfer exact final output to recipient. Enforce maximum input amount. Refund excess input tokens. What this function cannot/should not do Cannot execute if deadline expired. Cannot execute if required input > amountInMaximum. Cannot use invalid path encoding. Cannot process in wrong direction. Cannot bypass token approvals.

Contract Name	Function	Category
SwapRouter.sol	unwrapWETH9(uint256 amountMinimum, address recipient)	Main invariant(s) Path must be valid (token-fee-token-fee-...). Actual input $\leq$ amountInMaximum. Output amount is exact (as specified). Swaps execute in reverse order for exact output. Excess input refunded to msg.sender. Protected by deadline and slippage checks. Protected by reentrancy lock. Access level Public/External (requires token approvals) What this function can do Unwrap WETH9 held by router to ETH. Transfer ETH to recipient. Enforce minimum amount to unwrap. Used in combination with swaps ending in WETH. What this function cannot/should not do Cannot unwrap if WETH balance $<$ amountMinimum. Cannot unwrap to address(0). Cannot unwrap more than router's WETH balance. Should not fail silently if transfer fails. Main invariant(s) Router's WETH balance $\geq$ amountMinimum. Exact WETH balance is unwrapped and sent. Recipient receives ETH equal to unwrapped amount. Router's WETH balance becomes 0 after unwrap. Access level Public/External (operates on router's balance)
SwapRouter.sol	sweepToken(address token, uint256 amountMinimum, address recipient)	What this function can do Transfer entire token balance from router to recipient. Enforce minimum amount to sweep. Used to recover tokens stuck in router. Clean up after swaps.

Contract Name	Function	Category
SwapRouter.sol	refundETH()	What this function cannot/should not do Cannot sweep if balance < amountMinimum. Cannot sweep to address(0). Should not sweep WETH (use unwrapWETH9). Should not be relied upon for critical functionality. Main invariant(s) Router's token balance $\geq$ amountMinimum. Entire token balance is transferred. Router's balance becomes 0 after sweep. Used for cleanup and recovery. Access level Public/External (operates on router's balance) What this function can do Transfer entire ETH balance from router back to msg.sender. Used when excess ETH was sent in multicall. Clean up after ETH-based swaps. What this function cannot/should not do Cannot selectively refund partial amounts. Cannot refund to arbitrary address (always msg.sender). Cannot fail if balance is 0 (noop). Main invariant(s) Refunds entire ETH balance to caller. Router's ETH balance becomes 0. Safe to call even with 0 balance. Used in multicall for cleanup. Access level Public/External (refunds to msg.sender)
SwapRouter.sol	multicall(bytes[] calldata data)	What this function can do Execute multiple function calls in a single transaction. Combine swaps, unwraps, sweeps, refunds. Atomic execution (all or nothing). Improve gas efficiency. Return array of results from each call.

Contract Name	Function	Category
		What this function cannot/should not do Cannot execute if any call in batch fails. Cannot bypass individual function restrictions. Cannot be used for reentrancy attacks (protected). Should not include duplicate or conflicting calls. Main invariant(s) All calls execute in order or all revert. Each call maintains its own invariants. No reentrancy across multicall. msg.sender preserved for all sub-calls. Return data corresponds to input order. Access level Public/External (individual calls maintain their access controls)

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
UniswapV3Pool.sol	ERC20 (Token Pairs)	User-deposited liquidity (token0 and token1), Uncollected swap fees (LP portion), Protocol fees (awaiting collection by factory owner), Tokens owed to liquidity providers from burned positions, Flash loan principal during callback execution

Contract	Asset Type	Custodied Assets
NonfungiblePositionManager.sol	ERC721 + ERC20	NFT tokens representing individual liquidity positions, Transient ERC20 tokens during mint/increase/decrease operations, Uncollected tokens owed to position holders (tracked but held in pool)
SwapRouter.sol	ERC20 (Transient)	Transient token custody during multi-hop swap routing, Excess input tokens pending refund to user, Output tokens before final transfer to recipient, WETH during wrap/unwrap operations, Dust tokens from rounding (can be swept)
UniswapV3Factory.sol	None (Registry Only)	Does not custody any assets, Only stores pool addresses and fee tier configurations, Controls protocol fee collection rights

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
UniswapV3Pool.sol	mint()	token0 + token1 (from user via callback)	Liquidity position (virtual)	Public (via callback)	Callback verification bypass, incorrect token amounts, price manipulation at boundaries, reentrancy during callback, MEV sandwich attacks



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
UniswapV3Pool.sol	burn()	Liquidity position (virtual)	— (tokens marked as owed)	Public (position owner)	No immediate risk (tokens not transferred yet), but sets up for collect()
UniswapV3Pool.sol	collect()	—	token0 + token1 (owed amounts)	Public (position owner)	Reentrancy risk, unauthorized collection if position tracking broken, rounding errors in owed calculation
UniswapV3Pool.sol	swap()	tokenIn (zeroForOne ? token0 : token1)	tokenOut (zeroForOne ? token1 : token0)	Public (via callback)	Front-running, sandwich attacks, price manipulation, callback verification bypass, sqrtPriceLimit bypass, slippage exploitation, reentrancy during callback
UniswapV3Pool.sol	swap()	tokenIn (zeroForOne ? token0 : token1)	tokenOut (zeroForOne ? token1 : token0)	Public (via callback)	Flash loan attack vectors, price oracle manipulation, callback reentrancy, fee payment bypass, insufficient balance verification, protocol insolvency if callback exploited



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
UniswapV3Pool.sol	initialize()	—	— (sets price)	Public (once only)	First-mover price manipulation, initialization front-running, incorrect sqrtPrice setting, griefing by initializing at extreme prices
UniswapV3Pool.sol	collectProtocol()	—	token0 + token1 (protocol fees)	Restricted (factory owner)	Owner key compromise leads to fee theft, no rate limiting on collections, centralization risk
NonfungiblePositionManager.sol	mint()	token0 + token1 (from user)	ERC721 NFT + liquidity	Public	Slippage exceeded, deadline expiration, incorrect tick range, token approval front-running, MEV exploitation, minimum amount checks bypassed
NonfungiblePositionManager.sol	increaseLiquidity()	token0 + token1 (additional)	— (increases position)	Restricted (NFT owner/approved)	Same as mint: slippage, deadline, MEV, but also authorization bypass if approval system broken



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
SwapRouter.sol	decreaseLiquidity()	— (burns liquidity)	— (tokens marked owed)	Restricted (NFT owner/approved)	Authorization bypass, deadline expiration, incorrect liquidity calculation, sets up collect() risk
SwapRouter.sol	burn()	ERC721 NFT	—	Restricted (NFT owner/approved)	Burning with remaining liquidity/owed tokens (should revert), authorization bypass, storage corruption
SwapRouter.sol	exactInputSingle()	tokenIn (exact amount)	tokenOut ($\geq$ min)	Public	Deadline expiration, slippage exceeded (output < min), front-running, price manipulation, callback exploitation, token approval issues
SwapRouter.sol	exactInput()	tokenIn (exact amount)	tokenOut final ($\geq$ min)	Public	Multi-hop complexity amplifies risks: path manipulation, intermediate token price changes, cumulative slippage, deadline expires mid-route, reentrancy across hops



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
SwapRouter.sol	exactOutputSingle()	tokenIn ($\leq$ max)	tokenOut (exact amount)	Public	Input exceeds maximum, deadline expiration, price manipulation to force max input, refund calculation errors, excess token refund bypass
SwapRouter.sol	exactOutputSingle()	tokenIn ($\leq$ max)	tokenOut final (exact)	Public	Same as exactOutputSingle but amplified across hops, reverse path calculation errors, input underflow, cumulative maximum exceeded
SwapRouter.sol	unwrapWETH9()	WETH (from router)	ETH (to recipient)	Public	Insufficient WETH balance, recipient address(0), ETH transfer failure, amount minimum not met, draining router WETH
SwapRouter.sol	sweepToken()	token (from router)	token (to recipient)	Public	Draining router balance, recipient address(0), minimum amount bypass, sweep critical tokens accidentally



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
SwapRouter.sol	refundETH()	ETH (from router)	ETH (to msg.sender)	Public	Draining router ETH, but limited risk since refunds to caller, griefing by forcing refunds
SwapRouter.sol	multicall()	Varies per sub-call	Varies per sub-call	Public	Reentrancy across multiple calls, atomicity assumptions broken, msg.sender confusion, conflicting operations in batch, deadline bypass



Closing Summary

In this report, we have considered the security of Fluxion-Exchange. We performed our audit according to the procedure described above.

No critical issues in Fluxion-Exchange, just 1 issue of Informational severity was found. The Fluxion Exchange Team acknowledged the mentioned issue.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

February 2026

For

fluxion



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com